

SephiroFlows - Digital Ocean Deployment Guide

Last Updated: November 2025

Target Environment: Production deployment on Digital Ocean

Application: Express.js Backend + React Frontend + PostgreSQL + Redis

Table of Contents

1. [Prerequisites](#)
 2. [Digital Ocean Droplet Setup](#)
 3. [Initial Server Configuration](#)
 4. [Installing Dependencies](#)
 5. [Uploading the Application](#)
 6. [Environment Configuration](#)
 7. [Database Setup](#)
 8. [Building and Running the Application](#)
 9. [Nginx Configuration](#)
 10. [SSL/HTTPS Setup](#)
 11. [Process Management with PM2](#)
 12. [Domain Configuration](#)
 13. [Monitoring and Logs](#)
 14. [Troubleshooting](#)
 15. [Updating the Application](#)
-

1. Prerequisites

Before you begin, ensure you have:

Required Accounts & Access

-  **Digital Ocean Account** - Sign up at [digitalocean.com](https://www.digitalocean.com/) (<https://www.digitalocean.com/>)
-  **Domain Name** - Registered domain (e.g., from Namecheap, GoDaddy, Cloudflare)
-  **SSH Key Pair** - For secure server access (we'll create this in the next step)
-  **Git Repository** - Your SephiroFlows code (GitHub, GitLab, or Bitbucket)

Local Tools

-  **Terminal/Command Line** - Terminal (Mac/Linux) or PowerShell/WSL (Windows)
-  **SSH Client** - Built into Mac/Linux, or PuTTY for Windows
-  **Git** - For code deployment

Required Information

- SMTP server details (if using email notifications)
- Any third-party API keys your workflows will use
- Database backup strategy (for production data)

2. Digital Ocean Droplet Setup

2.1 Creating an SSH Key (If you don't have one)

On Mac/Linux:

```
# Generate a new SSH key
ssh-keygen -t ed25519 -C "your_email@example.com"

# Press Enter to accept default location (~/.ssh/id_ed25519)
# Enter a secure passphrase when prompted

# Display your public key
cat ~/.ssh/id_ed25519.pub
```

On Windows (PowerShell):

```
# Generate a new SSH key
ssh-keygen -t ed25519 -C "your_email@example.com"

# Press Enter to accept default location (C:\Users\YourName\.ssh\id_ed25519)
# Enter a secure passphrase when prompted

# Display your public key
type $env:USERPROFILE\.ssh\id_ed25519.pub
```

2.2 Adding SSH Key to Digital Ocean

1. Log in to [Digital Ocean](https://cloud.digitalocean.com/) (<https://cloud.digitalocean.com/>)
2. Click on **Settings** (left sidebar)
3. Select **Security** tab
4. Click **Add SSH Key**
5. Paste your public key (from the `cat` or `type` command above)
6. Give it a name (e.g., "My MacBook Pro")
7. Click **Add SSH Key**

2.3 Creating the Droplet

1. Click **Create** → **Droplets** (top right)
2. **Choose Region:**
 - Select a region close to your users
 - Recommended: New York, San Francisco, London, Frankfurt
3. **Choose an Image:**
 - **Ubuntu 22.04 (LTS) x64** ← Recommended

4. Choose Size:

Development/Testing:

- **Basic Plan → Regular → \$12/month**
- 2 GB RAM / 1 vCPU / 50 GB SSD
- Good for: Testing, low-traffic staging environments

Small Production:

- **Basic Plan → Regular → \$24/month**
- 4 GB RAM / 2 vCPUs / 80 GB SSD
- Good for: Small teams, 100-500 workflow runs/day

Medium Production (Recommended):

- **Basic Plan → Regular → \$48/month**
- 8 GB RAM / 4 vCPUs / 160 GB SSD
- Good for: Growing teams, 1,000-10,000 workflow runs/day

High-Traffic Production:

- **General Purpose → \$96/month+**
- 16 GB RAM / 4 vCPUs / 100 GB SSD (Premium Intel/AMD)
- Good for: Enterprise, 10,000+ workflow runs/day

1. Choose Authentication:

- Select **SSH Key**
- Check the box next to the SSH key you added earlier

2. Finalize Details:

- **Hostname:** `sephiroflows-prod` (or your preferred name)
- **Tags:** `production` , `sephiroflows`
- **Backups:** ☒ Enable (Recommended - adds 20% to cost)
- **Monitoring:** ☒ Enable (Free)

3. Click **Create Droplet**

4. **Wait for provisioning** (1-2 minutes)

5. **Copy the IP address** from the droplet details page

3. Initial Server Configuration

3.1 Connect to Your Server

```
# Replace YOUR_SERVER_IP with your droplet's IP address
ssh root@YOUR_SERVER_IP

# Example:
# ssh root@165.227.123.45

# If prompted about authenticity, type 'yes'
```

What this does: Establishes a secure connection to your server as the root user.

3.2 Update System Packages

```
# Update package list
apt update

# Upgrade all packages
apt upgrade -y

# This may take 3-5 minutes
```

What this does: Updates your server with the latest security patches and software versions.

3.3 Create a Non-Root User

Why? Running applications as root is a security risk. We'll create a dedicated user.

```
# Create a new user called 'sephiro'
adduser sephiro

# You'll be prompted for:
# - Password (choose a strong one)
# - Full name (optional, press Enter)
# - Other details (press Enter to skip)

# Add user to sudo group (allows running commands with sudo)
usermod -aG sudo sephiro

# Copy SSH keys to new user
rsync --archive --chown=sephiro:sephiro ~/.ssh /home/sephiro
```

What this does: Creates a non-root user with sudo privileges and copies your SSH keys so you can log in.

3.4 Test New User Login

Open a NEW terminal window (keep the current one open) and test:

```
ssh sephiro@YOUR_SERVER_IP
```

If successful, you should be logged in as `sephiro`. **Keep this new session open** and close the root session.

3.5 Configure Firewall (UFW)

```
# Check if UFW is installed
sudo ufw status

# Allow SSH (IMPORTANT: Do this first!)
sudo ufw allow OpenSSH

# Allow HTTP and HTTPS
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

# Enable firewall
sudo ufw enable

# Confirm by typing 'y' and pressing Enter

# Verify status
sudo ufw status numbered
```

Expected output:

```
Status: active

To Action From
--
[ 1] OpenSSH ALLOW IN Anywhere
[ 2] 80/tcp ALLOW IN Anywhere
[ 3] 443/tcp ALLOW IN Anywhere
```

What this does: Configures the firewall to only allow SSH (22), HTTP (80), and HTTPS (443) traffic.

3.6 Configure Timezone

```
# Set timezone (adjust to your location)
sudo timedatectl set-timezone America/New_York

# Verify
timedatectl

# List all available timezones:
# timedatectl list-timezones
```

What this does: Sets the correct timezone for log timestamps and scheduled tasks.

3.7 Configure Automatic Security Updates

```
# Install unattended-upgrades
sudo apt install unattended-upgrades -y

# Enable automatic security updates
sudo dpkg-reconfigure -plow unattended-upgrades

# Select 'Yes' when prompted
```

What this does: Automatically installs critical security updates, keeping your server secure.

4. Installing Dependencies

4.1 Install Node.js 20+

```
# Install Node.js 20 using NodeSource repository
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -

sudo apt install -y nodejs

# Verify installation
node --version
# Should show: v20.x.x

npm --version
# Should show: 10.x.x or higher
```

What this does: Installs Node.js 20 (required for the backend API) and npm (package manager).

4.2 Install PostgreSQL 15

```
# Add PostgreSQL APT repository
sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg
main" > /etc/apt/sources.list.d/pgdg.list'

# Import repository signing key
wget --quiet -O - https://www.postgresql.org/media/keys/ACCC4CF8.asc | sudo apt-key ad
d -

# Update package list
sudo apt update

# Install PostgreSQL 15
sudo apt install postgresql-15 postgresql-contrib-15 -y

# Verify installation
sudo -u postgres psql --version
# Should show: psql (PostgreSQL) 15.x
```

What this does: Installs PostgreSQL 15, the primary database for SephiroFlows.

4.3 Install Redis 7

```
# Add Redis repository
curl -fsSL https://packages.redis.io/gpg | sudo gpg --dearmor -o /usr/share/keyrings/
redis-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/redis-archive-keyring.gpg] https://
packages.redis.io/deb $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/
redis.list

# Update package list
sudo apt update

# Install Redis
sudo apt install redis -y

# Start Redis
sudo systemctl start redis-server

# Enable Redis to start on boot
sudo systemctl enable redis-server

# Verify installation
redis-cli ping
# Should respond: PONG
```

What this does: Installs Redis 7 for session storage and future queue management.

4.4 Install Nginx

```
# Install Nginx
sudo apt install nginx -y

# Start Nginx
sudo systemctl start nginx

# Enable Nginx to start on boot
sudo systemctl enable nginx

# Verify installation
sudo systemctl status nginx
# Should show: active (running)

# Test by visiting your server IP in a browser
# http://YOUR_SERVER_IP
# You should see the Nginx welcome page
```

What this does: Installs Nginx web server to act as a reverse proxy for your application.

4.5 Install PM2 (Process Manager)

```
# Install PM2 globally
sudo npm install -g pm2

# Verify installation
pm2 --version
# Should show version number (e.g., 5.3.0)

# Configure PM2 to start on boot
pm2 startup systemd -u sephiro --hp /home/sephiro

# Copy and run the command that PM2 outputs
# It will look like: sudo env PATH=$PATH:/usr/bin /usr/lib/node_modules/pm2/bin/pm2
startup systemd -u sephiro --hp /home/sephiro
```

What this does: Installs PM2, which will keep your Node.js application running and restart it if it crashes.

4.6 Install Git

```
# Install Git
sudo apt install git -y

# Verify installation
git --version
# Should show: git version 2.x.x

# Configure Git (optional, but recommended)
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

What this does: Installs Git for deploying code from your repository.

4.7 Install Certbot (for SSL/HTTPS)

```
# Install Certbot and Nginx plugin
sudo apt install certbot python3-certbot-nginx -y

# Verify installation
certbot --version
# Should show version number
```

What this does: Installs Certbot to obtain and manage free SSL certificates from Let's Encrypt.

5. Uploading the Application

You have three options to get your code onto the server:

Option 1: Using Git (Recommended for Production)

Best for: Continuous deployment, version control, team collaboration

```
# Create application directory
sudo mkdir -p /var/www/sephiroflows
sudo chown sephiro:sephiro /var/www/sephiroflows

# Navigate to directory
cd /var/www/sephiroflows

# Clone your repository
# For GitHub private repo:
git clone https://github.com/yourusername/sephiroflows.git .

# You'll be prompted for credentials
# Use Personal Access Token instead of password
# Create token at: https://github.com/settings/tokens

# For GitHub using SSH (if you've added deploy key):
git clone git@github.com:yourusername/sephiroflows.git .

# Verify files
ls -la
# You should see: backend, frontend, docker-compose.yml, etc.
```

Setting up Deploy Keys (Optional but Recommended):

```
# On the server, generate a deploy key
ssh-keygen -t ed25519 -C "sephiroflows-deploy" -f ~/.ssh/sephiroflows_deploy

# Display the public key
cat ~/.ssh/sephiroflows_deploy.pub

# Copy the output and add it to GitHub:
# 1. Go to your repo → Settings → Deploy keys
# 2. Click "Add deploy key"
# 3. Paste the key
# 4. Give it a title: "Production Server"
# 5. Check "Allow write access" if you want to push from server (not recommended)
# 6. Click "Add key"

# Configure Git to use the deploy key
nano ~/.ssh/config

# Add this configuration:
Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/sephiroflows_deploy
  IdentitiesOnly yes

# Save and exit (Ctrl+X, Y, Enter)

# Test connection
ssh -T git@github.com
# Should show: Hi username! You've successfully authenticated...
```

Option 2: Using SCP (Secure Copy)

Best for: Quick deployments, small updates

```
# From your LOCAL machine (not the server):
# Navigate to your project directory
cd /path/to/your/sephiroflows

# Upload the entire project
scp -r . sephiro@YOUR_SERVER_IP:/var/www/sephiroflows/

# This will take a few minutes depending on your internet speed

# Alternative: Create a tarball first (faster)
tar -czf sephiroflows.tar.gz --exclude=node_modules --exclude=.git .
scp sephiroflows.tar.gz sephiro@YOUR_SERVER_IP:/home/sephiro/

# Then on the server:
ssh sephiro@YOUR_SERVER_IP
sudo mkdir -p /var/www/sephiroflows
sudo chown sephiro:sephiro /var/www/sephiroflows
tar -xzf ~/sephiroflows.tar.gz -C /var/www/sephiroflows/
rm ~/sephiroflows.tar.gz
```

Option 3: Using rsync (Best for Updates)

Best for: Incremental updates, only transfers changed files

```
# From your LOCAL machine:
rsync -avz --exclude 'node_modules' --exclude '.git' \
  . sephiro@YOUR_SERVER_IP:/var/www/sephiroflows/

# Flags explained:
# -a: Archive mode (preserves permissions, timestamps)
# -v: Verbose (shows progress)
# -z: Compress during transfer
```

5.1 Set Correct Permissions

```
# On the server
cd /var/www/sephiroflows

# Set ownership
sudo chown -R sephiro:sephiro .

# Set directory permissions
find . -type d -exec chmod 755 {} \;

# Set file permissions
find . -type f -exec chmod 644 {} \;
```

6. Environment Configuration

6.1 Create Backend Environment File

```
cd /var/www/sephiroflows/backend  
  
# Create .env file  
nano .env
```

Add the following configuration (adjust values as needed):

```

# =====
# SERVER CONFIGURATION
# =====
NODE_ENV=production
PORT=3000
API_BASE_URL=https://yourdomain.com/api

# =====
# DATABASE CONFIGURATION
# =====
DB_HOST=localhost
DB_PORT=5432
DB_NAME=sephiroflows_prod
DB_USER=sephiro_user
DB_PASSWORD=YOUR_SECURE_DB_PASSWORD_HERE

# Connection Pool
DB_POOL_MIN=2
DB_POOL_MAX=10

# =====
# REDIS CONFIGURATION
# =====
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=
# Leave empty if no password is set (default local Redis)
# For production, consider setting a password

# =====
# AUTHENTICATION
# =====
JWT_SECRET=YOUR_SUPER_SECRET_JWT_KEY_HERE_MIN_32_CHARS
JWT_EXPIRES_IN=1h
SESSION_SECRET=YOUR_SESSION_SECRET_HERE_MIN_32_CHARS

# =====
# ENCRYPTION
# =====
# For encrypting sensitive data (credentials, API keys)
ENCRYPTION_KEY=YOUR_256_BIT_ENCRYPTION_KEY_HEX_64_CHARS

# Generate with: node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

# =====
# EMAIL CONFIGURATION (SMTP)
# =====
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=your-email@gmail.com
SMTP_PASSWORD=your-app-password
SMTP_FROM_NAME=SephiroFlows
SMTP_FROM_EMAIL=noreply@yourdomain.com

# For Gmail: Use App Password (https://myaccount.google.com/apppasswords)
# For SendGrid: smtp.sendgrid.net, port 587, username: apikey, password: YOUR_API_KEY
# For Mailgun: smtp.mailgun.org, port 587

# =====
# CORS CONFIGURATION

```

```
# =====
CORS_ORIGIN=https://yourdomain.com
# For multiple origins: https://yourdomain.com,https://app.yourdomain.com

# =====
# LOGGING
# =====
LOG_LEVEL=info
# Options: error, warn, info, debug

# =====
# RATE LIMITING
# =====
RATE_LIMIT_WINDOW_MS=900000
# 15 minutes in milliseconds
RATE_LIMIT_MAX_REQUESTS=100
# Max requests per window per IP

# =====
# OPTIONAL: EXTERNAL INTEGRATIONS
# =====
# Add API keys for integrations used in your workflows
# SLACK_CLIENT_ID=
# SLACK_CLIENT_SECRET=
# GOOGLE_CLIENT_ID=
# GOOGLE_CLIENT_SECRET=
```

Save and exit: Press `Ctrl+X`, then `Y`, then `Enter`

6.2 Generate Secure Secrets

Generate JWT Secret:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
# Copy the output and paste it as JWT_SECRET in .env
```

Generate Session Secret:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
# Copy and use as SESSION_SECRET
```

Generate Encryption Key:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
# Copy and use as ENCRYPTION_KEY
```

Generate Strong Database Password:

```
openssl rand -base64 32
# Copy and use as DB_PASSWORD
```

6.3 Secure the Environment File

```
# Set strict permissions (only owner can read/write)
chmod 600 .env

# Verify
ls -la .env
# Should show: -rw----- (600)
```

What this does: Ensures that only the `sephiro` user can read the sensitive configuration.

6.4 Create Frontend Environment File

```
cd /var/www/sephiroflows/frontend

# Create .env file
nano .env
```

Add:

```
# =====
# API CONFIGURATION
# =====
VITE_API_BASE_URL=https://yourdomain.com/api

# =====
# APPLICATION CONFIGURATION
# =====
VITE_APP_NAME=SephiroFlows
VITE_APP_VERSION=1.0.0

# =====
# FEATURE FLAGS (Optional)
# =====
# VITE_ENABLE_ANALYTICS=false
# VITE_ENABLE_DEBUG=false
```

Save and exit: `Ctrl+X`, `Y`, `Enter`

7. Database Setup

7.1 Secure PostgreSQL Installation

```
# Switch to postgres user
sudo -i -u postgres

# Access PostgreSQL prompt
psql
```

In the PostgreSQL prompt:

```
-- Change default postgres user password
ALTER USER postgres WITH PASSWORD 'your_secure_postgres_password';

-- Exit
❏q
```

Configure PostgreSQL to require password:

```
# Still as postgres user
nano /etc/postgresql/15/main/pg_hba.conf
```

Find this line:

```
local    all             postgres                                peer
```

Change to:

```
local    all             postgres                                md5
```

Save and exit: Ctrl+X, Y, Enter

```
# Restart PostgreSQL
exit
sudo systemctl restart postgresql
```

7.2 Create Database and User

```
# Connect as postgres
sudo -u postgres psql
```

In PostgreSQL prompt:

```
-- Create database
CREATE DATABASE sephiroflows_prod;

-- Create user (use the password from your backend .env)
CREATE USER sephiro_user WITH ENCRYPTED PASSWORD 'YOUR_SECURE_DB_PASSWORD_HERE';

-- Grant privileges
GRANT ALL PRIVILEGES ON DATABASE sephiroflows_prod TO sephiro_user;

-- Grant schema privileges
c sephiroflows_prod
GRANT ALL ON SCHEMA public TO sephiro_user;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO sephiro_user;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO sephiro_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON TABLES TO sephiro_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON SEQUENCES TO sephiro_user;

-- Verify connection
conninfo

-- List databases
l

-- Exit
q
```

7.3 Configure PostgreSQL for Production

```
# Edit PostgreSQL configuration
sudo nano /etc/postgresql/15/main/postgresql.conf
```

Find and update these settings (use `/` to search):

```
# Memory Settings (for 4GB RAM droplet - adjust based on your size)
shared_buffers = 1GB           # 25% of RAM
effective_cache_size = 3GB     # 75% of RAM
maintenance_work_mem = 256MB
work_mem = 16MB

# Checkpoint Settings
checkpoint_completion_target = 0.9
wal_buffers = 16MB
default_statistics_target = 100

# Query Planning
random_page_cost = 1.1        # For SSD storage
effective_io_concurrency = 200

# Logging (helpful for debugging)
logging_collector = on
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
log_rotation_age = 1d
log_rotation_size = 100MB
log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h '
log_timezone = 'UTC'

# Connection Settings
max_connections = 100
```

Save and exit: Ctrl+X, Y, Enter

Restart PostgreSQL:

```
sudo systemctl restart postgresql

# Verify it's running
sudo systemctl status postgresql
```

7.4 Run Database Migrations

```
cd /var/www/sephiroflows/backend

# Install dependencies (if not done yet)
npm install

# Run migrations
npm run migrate
```

Expected output:

```
Running migrations...
✓ Migration 001_initial_schema.sql applied
✓ Migration 002_add_indexes.sql applied
All migrations completed successfully
```

7.5 Set Up Database Backups

```
# Create backup directory
sudo mkdir -p /var/backups/sephiroflows
sudo chown sephiro:sephiro /var/backups/sephiroflows

# Create backup script
nano ~/backup-db.sh
```

Add this script:

```
#!/bin/bash

# Configuration
DB_NAME="sephiroflows_prod"
DB_USER="sephiro_user"
BACKUP_DIR="/var/backups/sephiroflows"
DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="$BACKUP_DIR/sephiroflows_$DATE.sql.gz"
RETENTION_DAYS=7

# Create backup
echo "Starting backup of $DB_NAME..."
PGPASSWORD="YOUR_SECURE_DB_PASSWORD_HERE" pg_dump -U $DB_USER -h localhost $DB_NAME |
gzip > $BACKUP_FILE

# Check if backup was successful
if [ $? -eq 0 ]; then
    echo "Backup completed successfully: $BACKUP_FILE"

    # Remove old backups
    find $BACKUP_DIR -name "sephiroflows_*.sql.gz" -mtime +$RETENTION_DAYS -delete
    echo "Old backups removed (older than $RETENTION_DAYS days)"
else
    echo "Backup failed!"
    exit 1
fi
```

Make executable:

```
chmod +x ~/backup-db.sh
```

Test the backup:

```
~/backup-db.sh
```

Schedule daily backups with cron:

```
crontab -e

# Select nano as editor if prompted

# Add this line (runs at 2 AM every day):
0 2 * * * /home/sephiro/backup-db.sh >> /home/sephiro/backup.log 2>&1

# Save and exit
```

8. Building and Running the Application

8.1 Install Backend Dependencies

```
cd /var/www/sephiroflows/backend

# Install production dependencies
npm install --production

# Or install all dependencies (if you need dev tools):
npm install
```

8.2 Build Backend (TypeScript Compilation)

```
# Compile TypeScript to JavaScript
npm run build

# Verify build
ls -la dist/

# You should see compiled .js files
```

8.3 Install Frontend Dependencies

```
cd /var/www/sephiroflows/frontend

# Install dependencies
npm install
```

8.4 Build Frontend (Production Build)

```
# Create optimized production build
npm run build

# This creates a 'dist' folder with static files
ls -la dist/

# You should see: index.html, assets/, etc.
```

What this does: Vite compiles and optimizes your React app into static HTML, CSS, and JavaScript files.

8.5 Test Backend Locally

```
cd /var/www/sephiroflows/backend  
  
# Start the backend (test mode)  
npm start  
  
# You should see:  
# Server running on port 3000  
# Database connected successfully
```

In another terminal, test the API:

```
curl http://localhost:3000/api/health  
  
# Expected response:  
# {"status":"ok","timestamp":"..."}
```

Press `Ctrl+C` to stop the server (we'll use PM2 next)

9. Nginx Configuration

9.1 Create Nginx Server Block

```
# Create configuration file  
sudo nano /etc/nginx/sites-available/sephiroflows
```

Add this configuration:

```

# =====
# SephiroFlows - Nginx Configuration
# =====

# Redirect HTTP to HTTPS (will be uncommented after SSL setup)
# server {
#     listen 80;
#     listen [::]:80;
#     server_name yourdomain.com www.yourdomain.com;
#     return 301 https://$server_name$request_uri;
# }

# HTTP Server (temporary - for testing before SSL)
server {
    listen 80;
    listen [::]:80;
    server_name yourdomain.com www.yourdomain.com;

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "no-referrer-when-downgrade" always;

    # Logs
    access_log /var/log/nginx/sephiroflows_access.log;
    error_log /var/log/nginx/sephiroflows_error.log;

    # Max body size (for file uploads in workflows)
    client_max_body_size 50M;

    # Backend API (reverse proxy to Express)
    location /api {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;

        # Proxy headers
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Timeouts (for long-running workflows)
        proxy_connect_timeout 60s;
        proxy_send_timeout 300s;
        proxy_read_timeout 300s;

        # Buffering
        proxy_buffering off;
        proxy_cache_bypass $http_upgrade;
    }

    # Webhooks (no authentication required)
    location /webhook {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

```

```

    proxy_set_header X-Forwarded-Proto $scheme;

    # Longer timeout for webhook processing
    proxy_read_timeout 300s;
}

# Frontend (serve static files)
location / {
    root /var/www/sephiroflows/frontend/dist;
    index index.html;
    try_files $uri $uri/ /index.html;

    # Cache static assets
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}

# Health check endpoint (for monitoring)
location /health {
    access_log off;
    return 200 "OK\n";
    add_header Content-Type text/plain;
}

# Deny access to hidden files
location ~ /\. {
    deny all;
    access_log off;
    log_not_found off;
}
}

# HTTPS Server (will be configured by Certbot)
# server {
#     listen 443 ssl http2;
#     listen [::]:443 ssl http2;
#     server_name yourdomain.com www.yourdomain.com;
#
#     # SSL certificates (will be added by Certbot)
#     # ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
#     # ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;
#
#     # ... (rest of configuration same as HTTP)
# }

```

Important: Replace `yourdomain.com` with your actual domain name in 4 places.

Save and exit: Ctrl+X, Y, Enter

9.2 Enable the Site

```
# Create symbolic link to enable the site
sudo ln -s /etc/nginx/sites-available/sephiroflows /etc/nginx/sites-enabled/

# Remove default site (optional)
sudo rm /etc/nginx/sites-enabled/default

# Test Nginx configuration
sudo nginx -t

# Expected output:
# nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
# nginx: configuration file /etc/nginx/nginx.conf test is successful
```

9.3 Reload Nginx

```
sudo systemctl reload nginx

# Verify it's running
sudo systemctl status nginx
```

10. SSL/HTTPS Setup

10.1 Point Your Domain to the Server

Before running Certbot, your domain must point to your server.

1. Log in to your domain registrar (Namecheap, GoDaddy, etc.)
2. Go to DNS settings for your domain
3. Add/edit these records:

Type	Name	Value	TTL
----	----	-----	---
A	@	YOUR_SERVER_IP	300
A	www	YOUR_SERVER_IP	300

1. **Wait 5-30 minutes** for DNS propagation

Test DNS propagation:

```
# From your local machine or server
dig yourdomain.com

# Or use online tools:
# https://dnschecker.org/
```

10.2 Obtain SSL Certificate with Certbot

```
# Run Certbot with Nginx plugin
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com

# You'll be prompted for:
# 1. Email address (for urgent renewal and security notices)
# 2. Agree to Terms of Service (Y)
# 3. Share email with EFF (optional, N is fine)
# 4. Redirect HTTP to HTTPS? (Choose 2: Redirect)
```

Expected output:

```
Congratulations! Your certificate and chain have been saved at:
/etc/letsencrypt/live/yourdomain.com/fullchain.pem
Your key file has been saved at:
/etc/letsencrypt/live/yourdomain.com/privkey.pem
Your cert will expire on 2026-02-XX. To obtain a new or tweaked
version of this certificate in the future, simply run certbot again
with the "certonly" option.
```

What this does:

- Obtains a free SSL certificate from Let's Encrypt
- Automatically configures Nginx to use HTTPS
- Sets up automatic certificate renewal

10.3 Verify SSL Configuration

```
# Test Nginx configuration
sudo nginx -t

# Reload Nginx
sudo systemctl reload nginx
```

Test in browser:

1. Visit `https://yourdomain.com`
2. You should see a padlock icon in the address bar
3. Click the padlock → Certificate should be valid

Test SSL rating:

Visit: <https://www.ssllabs.com/ssltest/analyze.html?d=yourdomain.com>

10.4 Configure Automatic Certificate Renewal

```
# Test renewal (dry run)
sudo certbot renew --dry-run

# Expected output:
# Congratulations, all simulated renewals succeeded
```

Certbot automatically sets up a cron job or systemd timer for renewal.

Verify automatic renewal:

```
# Check systemd timer
sudo systemctl list-timers | grep certbot

# Or check cron
sudo cat /etc/cron.d/certbot
```

10.5 Enhance SSL Security (Optional but Recommended)

```
# Edit Nginx SSL configuration
sudo nano /etc/nginx/sites-available/sephiroflows
```

Find the HTTPS server block (added by Certbot) and add these lines inside:

```
server {
    listen 443 ssl http2;
    # ... existing SSL configuration ...

    # Enhanced SSL Security
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers off;
    ssl_ciphers 'ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-
    ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384';

    # SSL session cache
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 10m;
    ssl_session_tickets off;

    # HSTS (HTTP Strict Transport Security)
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; pre-
    load" always;

    # OCSP Stapling
    ssl_stapling on;
    ssl_stapling_verify on;
    ssl_trusted_certificate /etc/letsencrypt/live/yourdomain.com/chain.pem;
    resolver 8.8.8.8 8.8.4.4 valid=300s;
    resolver_timeout 5s;

    # ... rest of your configuration ...
}
```

Save, test, and reload:

```
sudo nginx -t
sudo systemctl reload nginx
```

11. Process Management with PM2

11.1 Create PM2 Ecosystem File

```
cd /var/www/sephiroflows

# Create ecosystem configuration
nano ecosystem.config.js
```

Add this configuration:

```
module.exports = {
  apps: [
    {
      name: 'sephiroflows-api',
      cwd: '/var/www/sephiroflows/backend',
      script: './dist/index.js',
      instances: 2, // Adjust based on CPU cores (e.g., 2 for 2 vCPUs)
      exec_mode: 'cluster',
      env: {
        NODE_ENV: 'production',
      },
      error_file: '/var/www/sephiroflows/logs/pm2-api-error.log',
      out_file: '/var/www/sephiroflows/logs/pm2-api-out.log',
      log_date_format: 'YYYY-MM-DD HH:mm:ss Z',
      merge_logs: true,

      // Restart configuration
      autorestart: true,
      max_restarts: 10,
      min_uptime: '10s',
      max_memory_restart: '500M',

      // Graceful shutdown
      kill_timeout: 5000,
      wait_ready: true,
      listen_timeout: 10000,

      // Health monitoring
      watch: false, // Don't watch in production
      ignore_watch: ['node_modules', 'logs'],
    },
  ],
};
```

Save and exit: Ctrl+X, Y, Enter

11.2 Create Logs Directory

```
mkdir -p /var/www/sephiroflows/logs
```

11.3 Start Application with PM2

```
cd /var/www/sephiroflows

# Start applications
pm2 start ecosystem.config.js
```

You should see:

```
#
#
```

	id	name	status	restart	uptime	cpu
#	0	sephiroflows-api	online	0	0s	0%
#	1	sephiroflows-api	online	0	0s	0%

```
#
```

11.4 Save PM2 Process List

```
# Save current process list
pm2 save
```

This ensures PM2 restarts these apps on server reboot

11.5 Verify Startup Script

```
# Verify PM2 startup is configured
pm2 startup
```

If not configured, run the command it provides

11.6 Test the Application

Test API:

```
curl https://yourdomain.com/api/health

# Expected response:
# {"status":"ok","timestamp":"2025-11-28T..."}
```

Test Frontend:

Open browser and visit: <https://yourdomain.com>

You should see the SephiroFlows login/landing page.

11.7 PM2 Commands Cheat Sheet

```
# View running processes
pm2 list

# View detailed info
pm2 info sephiroflows-api

# View logs (all apps)
pm2 logs

# View logs (specific app)
pm2 logs sephiroflows-api

# View last 100 lines
pm2 logs --lines 100

# Stream logs in real-time
pm2 logs --raw

# Monitor resources
pm2 monit

# Restart app
pm2 restart sephiroflows-api

# Reload app (zero-downtime restart)
pm2 reload sephiroflows-api

# Stop app
pm2 stop sephiroflows-api

# Delete app from PM2
pm2 delete sephiroflows-api

# Restart all apps
pm2 restart all

# Clear all logs
pm2 flush
```

12. Domain Configuration

12.1 DNS Configuration Summary

Your DNS records should look like this:

Type	Name	Value	TTL	Purpose
----	----	-----	---	-----
A	@	YOUR_SERVER_IP	300	Root domain
A	www	YOUR_SERVER_IP	300	WWW subdomain
AAAA	@	YOUR_IPV6_IP	300	IPv6 (optional)
AAAA	www	YOUR_IPV6_IP	300	IPv6 WWW (optional)

12.2 Subdomain Configuration (Optional)

If you want separate subdomains for API and app:

Example: `app.yourdomain.com` for frontend, `api.yourdomain.com` for backend

DNS Records:

Type	Name	Value	TTL
----	----	-----	---
A	app	YOUR_SERVER_IP	300
A	api	YOUR_SERVER_IP	300

Update Nginx Configuration:

```
sudo nano /etc/nginx/sites-available/sephiroflows
```

Create separate server blocks:

```
# API subdomain
server {
    listen 443 ssl http2;
    server_name api.yourdomain.com;

    # SSL config...

    location / {
        proxy_pass http://localhost:3000;
        # ... proxy settings ...
    }
}

# App subdomain
server {
    listen 443 ssl http2;
    server_name app.yourdomain.com;

    # SSL config...

    location / {
        root /var/www/sephiroflows/frontend/dist;
        try_files $uri /index.html;
    }
}
```

Obtain SSL for subdomains:

```
sudo certbot --nginx -d api.yourdomain.com -d app.yourdomain.com
```

12.3 Update Frontend Environment

```
nano /var/www/sephiroflows/frontend/.env
```

Update API URL:

```
VITE_API_BASE_URL=https://api.yourdomain.com
# Or if using single domain: https://yourdomain.com/api
```

Rebuild frontend:

```
cd /var/www/sephiroflows/frontend  
npm run build
```

13. Monitoring and Logs

13.1 Application Logs (PM2)

```
# View all logs  
pm2 logs  
  
# View last 200 lines  
pm2 logs --lines 200  
  
# View only errors  
pm2 logs --err  
  
# Save logs to file  
pm2 logs --out /home/sephiro/app-logs.txt
```

13.2 Nginx Logs

```
# Access logs (all requests)  
sudo tail -f /var/log/nginx/sephiroflows_access.log  
  
# Error logs  
sudo tail -f /var/log/nginx/sephiroflows_error.log  
  
# View last 50 errors  
sudo tail -n 50 /var/log/nginx/sephiroflows_error.log  
  
# Search for specific error  
sudo grep "502 Bad Gateway" /var/log/nginx/sephiroflows_error.log
```

13.3 PostgreSQL Logs

```
# Find log directory  
sudo -u postgres psql -c "SHOW log_directory;"  
  
# View logs  
sudo tail -f /var/log/postgresql/postgresql-15-main.log  
  
# Or if in data directory:  
sudo tail -f /var/lib/postgresql/15/main/log/postgresql-*.log
```

13.4 Redis Logs

```
# View Redis logs
sudo tail -f /var/log/redis/redis-server.log

# Monitor Redis in real-time
redis-cli monitor
```

13.5 System Logs

```
# View system logs
sudo journalctl -xe

# View logs for specific service
sudo journalctl -u nginx
sudo journalctl -u postgresql
sudo journalctl -u redis

# Follow logs in real-time
sudo journalctl -f
```

13.6 Set Up Log Rotation

For PM2 logs:

```
# Install PM2 log rotate module
pm2 install pm2-logrotate

# Configure rotation
pm2 set pm2-logrotate:max_size 10M
pm2 set pm2-logrotate:retain 7
pm2 set pm2-logrotate:compress true
```

For application logs:

```
sudo nano /etc/logrotate.d/sephiroflows
```

Add:

```
/var/www/sephiroflows/logs/*.log {
    daily
    missingok
    rotate 14
    compress
    delaycompress
    notifempty
    create 0644 sephiro sephiro
    sharedscripts
}
```

13.7 Set Up Basic Monitoring

Install monitoring tools:

```
# Install htop for process monitoring
sudo apt install htop -y

# Install iotop for disk I/O monitoring
sudo apt install iotop -y

# Install nethogs for network monitoring
sudo apt install nethogs -y
```

Quick monitoring commands:

```
# View resource usage
htop

# View disk I/O
sudo iotop

# View network usage
sudo nethogs

# View disk usage
df -h

# View memory usage
free -h

# View PostgreSQL connections
sudo -u postgres psql -c "SELECT count(*) FROM pg_stat_activity;"

# View Redis memory
redis-cli info memory
```

13.8 Digital Ocean Monitoring (Built-in)

1. Go to your droplet in Digital Ocean dashboard
2. Click **Graphs** tab
3. View real-time metrics:
 - CPU usage
 - Memory usage
 - Disk I/O
 - Network bandwidth

Set up alerts:

1. Click **Create Alert Policy**
2. Configure alerts for:
 - CPU > 80% for 5 minutes
 - Memory > 90% for 5 minutes
 - Disk > 90%
3. Enter email for notifications

13.9 Application Health Check Script

```
nano ~/health-check.sh
```

Add:

```
#!/bin/bash

echo "=====
echo "SephroFlows Health Check"
echo "=====
echo ""

# Check API
echo "1. Checking API..."
API_STATUS=$(curl -s -o /dev/null -w "%{http_code}" https://yourdomain.com/api/health)
if [ "$API_STATUS" = "200" ]; then
    echo "    ✓ API is healthy (200 OK)"
else
    echo "    x API is down (HTTP $API_STATUS)"
fi

# Check frontend
echo "2. Checking Frontend..."
FRONTEND_STATUS=$(curl -s -o /dev/null -w "%{http_code}" https://yourdomain.com)
if [ "$FRONTEND_STATUS" = "200" ]; then
    echo "    ✓ Frontend is healthy (200 OK)"
else
    echo "    x Frontend is down (HTTP $FRONTEND_STATUS)"
fi

# Check PM2
echo "3. Checking PM2 processes..."
pm2 list | grep online > /dev/null
if [ $? -eq 0 ]; then
    echo "    ✓ PM2 processes are running"
    pm2 jlist | grep -o '"status": "[^"]*"' | sort | uniq -c
else
    echo "    x No PM2 processes running"
fi

# Check PostgreSQL
echo "4. Checking PostgreSQL..."
sudo -u postgres psql -c "SELECT 1;" > /dev/null 2>&1
if [ $? -eq 0 ]; then
    echo "    ✓ PostgreSQL is running"
else
    echo "    x PostgreSQL is down"
fi

# Check Redis
echo "5. Checking Redis..."
redis-cli ping > /dev/null 2>&1
if [ $? -eq 0 ]; then
    echo "    ✓ Redis is running"
else
    echo "    x Redis is down"
fi

# Check Nginx
echo "6. Checking Nginx..."
sudo systemctl is-active --quiet nginx
if [ $? -eq 0 ]; then
    echo "    ✓ Nginx is running"
else
    echo "    x Nginx is down"
fi
```

```
# Disk space
echo "7. Checking Disk Space..."
DISK_USAGE=$(df -h / | awk 'NR==2 {print $5}' | sed 's/%//')
if [ "$DISK_USAGE" -lt 80 ]; then
    echo "    ✓ Disk usage: ${DISK_USAGE}%"
else
    echo "    ⚠ WARNING: Disk usage is high: ${DISK_USAGE}%"
fi

# Memory
echo "8. Checking Memory..."
MEM_USAGE=$(free | grep Mem | awk '{printf "%.0f", $3/$2 * 100}')
if [ "$MEM_USAGE" -lt 90 ]; then
    echo "    ✓ Memory usage: ${MEM_USAGE}%"
else
    echo "    ⚠ WARNING: Memory usage is high: ${MEM_USAGE}%"
fi

echo ""
echo "=====
```

Make executable and run:

```
chmod +x ~/health-check.sh
~/health-check.sh
```

14. Troubleshooting

14.1 Application Won't Start

Check PM2 logs:

```
pm2 logs sephiroflows-api --lines 50
```

Common issues:

1. Database connection error:

```
Error: connect ECONNREFUSED 127.0.0.1:5432
```

Solution:

```
# Check PostgreSQL is running
sudo systemctl status postgresql

# Check database credentials in .env
nano /var/www/sephiroflows/backend/.env

# Test connection manually
psql -h localhost -U sephiro_user -d sephiroflows_prod
```

2. Port already in use:

```
Error: listen EADDRINUSE :::3000
```

Solution:

```
# Find process using port 3000
sudo lsof -i :3000

# Kill the process
sudo kill -9 <PID>

# Or change port in .env
```

3. Missing environment variables:

```
Error: JWT_SECRET is required
```

Solution:

```
# Check .env file exists and has correct values
cat /var/www/sephiroflows/backend/.env | grep JWT_SECRET
```

14.2 502 Bad Gateway Error

Meaning: Nginx can't connect to the backend.

Diagnosis:

```
# Check if backend is running
pm2 list

# Check backend logs
pm2 logs sephiroflows-api

# Check Nginx error log
sudo tail -n 50 /var/log/nginx/sephiroflows_error.log
```

Common causes:

1. Backend crashed → Restart: `pm2 restart sephiroflows-api`
2. Wrong port in Nginx config → Check `proxy_pass http://localhost:3000;`
3. Firewall blocking → Check: `sudo ufw status`

14.3 Frontend Shows Blank Page

Check browser console:

1. Open DevTools (F12)
2. Check Console tab for errors
3. Check Network tab for failed requests

Common issues:

1. API connection error:

```
Failed to load resource: net::ERR_CONNECTION_REFUSED
```

Solution:

- Check VITE_API_BASE_URL in frontend/.env
- Ensure backend is running
- Check CORS configuration in backend

2. Wrong base URL:

```
GET https://localhost:3000/api/... 404 Not Found
```

Solution:

```
# Update frontend .env
nano /var/www/sephiroflows/frontend/.env
# Change to: VITE_API_BASE_URL=https://yourdomain.com/api

# Rebuild frontend
cd /var/www/sephiroflows/frontend
npm run build
```

14.4 SSL Certificate Errors

“Your connection is not private” error:**Check certificate:**

```
# Test certificate
sudo certbot certificates

# Renew if needed
sudo certbot renew
```

Check Nginx configuration:

```
sudo nginx -t
sudo systemctl reload nginx
```

Force certificate renewal:

```
sudo certbot renew --force-renewal
sudo systemctl reload nginx
```

14.5 High CPU Usage

Identify culprit:

```
# Check PM2 processes
pm2 monit

# Check all processes
htop

# Sort by CPU (press P in htop)
```

Common causes:

1. Infinite loop in workflow
2. Too many concurrent executions
3. Database query without indexes
4. Node.js event loop blocked

Solutions:

- Scale down PM2 instances: Edit `ecosystem.config.js`
- Add database indexes
- Optimize workflow logic
- Upgrade droplet size

14.6 Database Connection Pool Exhausted

Error:

```
Error: Connection pool exhausted
```

Check connections:

```
sudo -u postgres psql -d sephiroflows_prod -c "
SELECT count(*) as total_connections,
       count(*) FILTER (WHERE state = 'active') as active,
       count(*) FILTER (WHERE state = 'idle') as idle
FROM pg_stat_activity
WHERE datname = 'sephiroflows_prod';
"
```

Solutions:

```
# Increase pool size in backend/.env
DB_POOL_MAX=20 # Increase from 10

# Or increase max_connections in PostgreSQL
sudo nano /etc/postgresql/15/main/postgresql.conf
# Set: max_connections = 200

sudo systemctl restart postgresql
```

14.7 Out of Memory Errors

Check memory usage:

```
free -h
# Shows total, used, free memory

# Check swap
swapon --show
```

Add swap space:

```
# Create 2GB swap file
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile

# Make permanent
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab

# Verify
free -h
```

Reduce memory usage:

```
# Reduce PM2 instances
nano /var/www/sephiroflows/ecosystem.config.js
# Change: instances: 1

pm2 restart all
```

14.8 Slow Database Queries

Enable slow query log:

```
sudo nano /etc/postgresql/15/main/postgresql.conf
```

Add:

```
log_min_duration_statement = 1000 # Log queries slower than 1 second
```

Restart:

```
sudo systemctl restart postgresql
```

View slow queries:

```
sudo tail -f /var/log/postgresql/postgresql-15-main.log | grep "duration:"
```

Analyze query:

```
sudo -u postgres psql -d sephiroflows_prod

# Run EXPLAIN on slow query
EXPLAIN ANALYZE SELECT ...;
```

Add indexes:

```
-- Example: Add index on tenant_id + created_at
CREATE INDEX idx_workflow_runs_tenant_created
ON workflow_runs(tenant_id, created_at DESC);
```

14.9 Disk Space Full

Check disk usage:

```
df -h

# Find large files
du -h /var/www/sephiroflows | sort -rh | head -20
```

Clean up:

```
# Clear PM2 logs
pm2 flush

# Clean old backups
rm /var/backups/sephiroflows/*_old.sql.gz

# Clean package caches
sudo apt-get clean
sudo apt-get autoclean

# Clean journal logs
sudo journalctl --vacuum-time=7d
```

15. Updating the Application

15.1 Prepare for Update

```
# Create a backup first!
~/backup-db.sh

# Or manual backup:
PGPASSWORD="YOUR_DB_PASSWORD" pg_dump -U sephiro_user -h localhost sephiroflows_prod
> ~/backup_before_update.sql
```

15.2 Update from Git

```
cd /var/www/sephiroflows

# Fetch latest changes
git fetch origin

# Check what's new
git log HEAD..origin/main --oneline

# Pull changes
git pull origin main
```

15.3 Update Backend

```
cd /var/www/sephiroflows/backend

# Install new dependencies
npm install

# Run new migrations (if any)
npm run migrate

# Rebuild TypeScript
npm run build

# Restart application (zero-downtime)
pm2 reload sephiroflows-api
```

15.4 Update Frontend

```
cd /var/www/sephiroflows/frontend

# Install new dependencies
npm install

# Rebuild
npm run build

# No restart needed - Nginx serves static files
```

15.5 Verify Update

```
# Check PM2 status
pm2 list

# Check logs for errors
pm2 logs --lines 50

# Run health check
~/health-check.sh

# Test in browser
curl https://yourdomain.com/api/health
```

15.6 Rollback if Needed

If something goes wrong:

```
# Stop application
pm2 stop sephiroflows-api

# Restore database
PGPASSWORD="YOUR_DB_PASSWORD" psql -U sephiro_user -h localhost sephiroflows_prod < ~/
backup_before_update.sql

# Rollback code
git reset --hard HEAD~1
# Or: git checkout <previous-commit-hash>

# Rebuild
cd backend
npm install
npm run build

cd ../frontend
npm install
npm run build

# Start application
pm2 restart sephiroflows-api

# Verify
pm2 logs
```

15.7 Automated Deployment Script

Create deployment script:

```
nano ~/deploy.sh
```

Add:

```
#!/bin/bash

set -e # Exit on error

echo "=====
echo "SephiroFlows Deployment Script"
echo "=====
echo ""

APP_DIR="/var/www/sephiroflows"
BACKUP_DIR="/var/backups/sephiroflows"

# 1. Backup database
echo "Step 1: Creating database backup..."
$HOME/backup-db.sh

# 2. Pull latest code
echo "Step 2: Pulling latest code..."
cd $APP_DIR
git pull origin main

# 3. Update backend
echo "Step 3: Updating backend..."
cd $APP_DIR/backend
npm install --production
npm run build

# 4. Run migrations
echo "Step 4: Running database migrations..."
npm run migrate

# 5. Update frontend
echo "Step 5: Updating frontend..."
cd $APP_DIR/frontend
npm install
npm run build

# 6. Reload application
echo "Step 6: Reloading application..."
pm2 reload sephiroflows-api

# 7. Verify
echo "Step 7: Verifying deployment..."
sleep 5
pm2 list

echo ""
echo "=====
echo "Deployment completed successfully!"
echo "=====

# Run health check
$HOME/health-check.sh
```

Make executable:

```
chmod +x ~/deploy.sh
```

Use it:

```
~/deploy.sh
```

Additional Resources

Useful Commands Quick Reference

```
# System
sudo reboot                # Reboot server
sudo shutdown -h now       # Shutdown server
df -h                     # Check disk space
free -h                   # Check memory
htop                      # Monitor processes

# Application
cd /var/www/sephiroflows   # Go to app directory
pm2 list                   # List PM2 processes
pm2 logs                   # View logs
pm2 restart sephiroflows-api # Restart app
pm2 monit                  # Monitor resources

# Database
sudo -u postgres psql      # Access PostgreSQL
psql -U sephiro_user -d sephiroflows_prod # Access app database
~/backup-db.sh             # Backup database

# Nginx
sudo nginx -t              # Test configuration
sudo systemctl reload nginx # Reload Nginx
sudo tail -f /var/log/nginx/sephiroflows_error.log # View errors

# SSL
sudo certbot certificates   # View certificates
sudo certbot renew          # Renew certificates

# Updates
~/deploy.sh                 # Deploy updates
~/health-check.sh           # Check system health
```

Important File Locations

```

Application:
/var/www/sephiroflows/           # Application root
/var/www/sephiroflows/backend/    # Backend code
/var/www/sephiroflows/frontend/  # Frontend code
/var/www/sephiroflows/logs/      # Application logs

Configuration:
/var/www/sephiroflows/backend/.env # Backend environment
/var/www/sephiroflows/frontend/.env # Frontend environment
/var/www/sephiroflows/ecosystem.config.js # PM2 config

Nginx:
/etc/nginx/sites-available/sephiroflows # Nginx config
/etc/nginx/sites-enabled/sephiroflows  # Enabled site
/var/log/nginx/sephiroflows_*.log     # Nginx logs

Database:
/etc/postgresql/15/main/postgresql.conf # PostgreSQL config
/var/lib/postgresql/15/main/           # Database files
/var/backups/sephiroflows/             # Database backups

SSL:
/etc/letsencrypt/live/yourdomain.com/  # SSL certificates

Scripts:
/home/sephiro/backup-db.sh             # Backup script
/home/sephiro/deploy.sh                 # Deployment script
/home/sephiro/health-check.sh          # Health check script

```

Getting Help

Digital Ocean Community: <https://www.digitalocean.com/community/questions>

PostgreSQL Documentation: <https://www.postgresql.org/docs/15/>

Nginx Documentation: <https://nginx.org/en/docs/>

PM2 Documentation: <https://pm2.keymetrics.io/docs/>

Node.js Documentation: <https://nodejs.org/docs/>

Let's Encrypt Community: <https://community.letsencrypt.org/>

Security Checklist

Before going live, ensure:

- ☐ SSH key authentication enabled
- ☐ Root login disabled
- ☐ Firewall (UFW) enabled with only necessary ports
- ☐ Non-root user created for running application
- ☐ PostgreSQL password authentication enabled
- ☐ Strong passwords for database users
- ☐ Environment files have 600 permissions
- ☐ SSL certificate installed and valid
- ☐ HTTPS redirect configured

- ☐ Security headers configured in Nginx
 - ☐ Automatic security updates enabled
 - ☐ Database backups automated and tested
 - ☐ Application logs configured
 - ☐ Monitoring and alerts set up
 - ☐ Rate limiting enabled in backend
 - ☐ CORS properly configured
 - ☐ Sensitive data encrypted in database
-

Performance Optimization Checklist

- ☐ Gzip compression enabled in Nginx
 - ☐ Static assets cached (CSS, JS, images)
 - ☐ Database indexes created for frequent queries
 - ☐ PostgreSQL configuration optimized for your RAM
 - ☐ Connection pooling configured
 - ☐ PM2 cluster mode enabled
 - ☐ Redis session storage configured
 - ☐ CDN configured for static assets (optional)
 - ☐ Database query optimization reviewed
 - ☐ Application bundling optimized (frontend)
-

Congratulations! 🎉 Your SephiroFlows application is now deployed on Digital Ocean!

Next steps:

1. Create your first workflow in the UI
2. Set up monitoring alerts
3. Configure any third-party integrations
4. Invite your team members
5. Review security settings

Need help? Review the Troubleshooting section or check the application logs with `pm2 logs`.

This guide is maintained as part of the SephiroFlows project. Last updated: November 2025